Attention Is All You Need

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, Illia Polosukhin

Google Brain; Google Research; University of Toronto

arXiv:1706.03762v1 | NeurIPS 2017

Abstract

We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. The base model, with 65 million parameters, trains in 12 hours on 8 NVIDIA P100 GPUs. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing.

Keywords: self-attention, transformer, machine translation, sequence transduction, multi-head attention

1 Introduction

Recurrent neural networks, long short-term memory networks and gated recurrent neural networks in particular, have been firmly established as state of the art approaches in sequence modeling and transduction problems such as language modeling and machine translation. Numerous efforts have since continued to push the boundaries of recurrent language models and encoder-decoder architectures. Recurrent models typically factor computation along the symbol positions of the input and output sequences. Aligning the positions to steps in computation time, they generate a sequence of hidden states h_t as a function of the previous hidden state h_(t-1) and the input for position t. This inherently sequential nature precludes parallelization within training examples, which becomes critical at longer sequence lengths, as memory constraints limit batching across examples.

Attention mechanisms have become an integral part of compelling sequence modeling and transduction models in various tasks, allowing modeling of dependencies without regard to their distance in the input or output sequences. In all but a few cases, however, such attention mechanisms are used in conjunction with a recurrent network. In this work we propose the Transformer, a model architecture eschewing recurrence and instead relying entirely on an attention mechanism to draw global dependencies between input and output. The Transformer allows for significantly more parallelization and can reach a new state of the art in translation quality after being trained for as little as twelve hours on eight P100 GPUs. Figure 1 summarizes the architecture, and every subsequent section refers back to it.

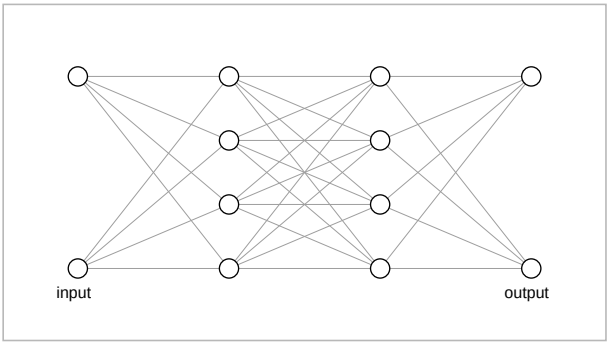


Figure 1: The Transformer architecture, with stacked self-attention and feed-forward layers in the encoder and decoder.

2 Background

The goal of reducing sequential computation also forms the foundation of the Extended Neural GPU, ByteNet and ConvS2S, all of which use convolutional neural networks as basic building blocks, computing hidden representations in parallel for all input and output positions. In these models, the number of operations required to relate signals from two arbitrary input or output positions grows in the distance between positions, linearly for ConvS2S and logarithmically for ByteNet. This makes it more difficult to learn dependencies between distant positions. In the Transformer this is reduced to a constant number of operations, albeit at the cost of reduced effective resolution due to averaging

attention-weighted positions, an effect we counteract with multi-head attention.

Self-attention, sometimes called intra-attention, is an attention mechanism relating different positions of a single sequence in order to compute a representation of the sequence. Self-attention has been used successfully in a variety of tasks including reading comprehension, abstractive summarization, textual entailment and learning task-independent sentence representations. End-to-end memory networks are based on a recurrent attention mechanism instead of sequence-aligned recurrence and have been shown to perform well on simple-language question answering and language modeling tasks. To the best of our knowledge, however, the Transformer is the first transduction model relying entirely on self-attention to compute representations of its input and output without using sequence-aligned RNNs or convolution.

3 Model Architecture

Most competitive neural sequence transduction models have an encoder-decoder structure. Here, the encoder maps an input sequence of symbol representations (x_1, ..., x_n) to a sequence of continuous representations z = (z_1, ..., z_n). Given z, the decoder then generates an output sequence (y_1, ..., y_m) of symbols one element at a time. At each step the model is auto-regressive, consuming the previously generated symbols as additional input when generating the next. The Transformer follows this overall architecture using stacked self-attention and point-wise, fully connected layers for both the encoder and decoder, shown in the left and right halves of Figure 1 respectively.

3.1 Encoder and Decoder Stacks

The encoder is composed of a stack of N = 6 identical layers. Each layer has two sub-layers. The first is a multi-head self-attention mechanism, and the second is a simple, position-wise fully connected feed-forward network. We employ a residual connection around each of the two sub-layers, followed by layer normalization. That is, the output of each sub-layer is LayerNorm(x + Sublayer(x)), where Sublayer(x) is the function implemented by the sub-layer itself. To facilitate these residual connections, all sub-layers in the model, as well as the embedding layers, produce outputs of dimension d_model = 512.

The decoder is also composed of a stack of N = 6 identical layers. In addition to the two sub-layers in each encoder layer, the decoder inserts a third sub-layer, which performs multi-head attention over the output of the encoder stack. Similar to the encoder, we employ residual connections around each of the sub-layers, followed by layer normalization. We also modify the self-attention sub-layer in the decoder stack to prevent positions from attending to subsequent positions. This masking, combined with the fact that the output embeddings are offset by one position, ensures that the predictions for position i can depend only on the known outputs at positions less than i.

3.2 Attention

An attention function can be described as mapping a query and a set of key-value pairs to an output, where the query, keys, values, and output are all vectors. The output is computed as a weighted sum of the values, where the weight assigned to each value is computed by a compatibility function of the query with the corresponding key. We describe two attention variants that are used throughout the Transformer: scaled dot-product attention and multi-head attention.

3.2.1 Scaled Dot-Product Attention

We call our particular attention scaled dot-product attention. The input consists of queries and keys of dimension d_k, and values of dimension d_v. We compute the dot products of the query with all keys, divide each by sqrt(d_k), and apply a softmax function to obtain the weights on the values. In practice, we compute the attention function on a set of queries simultaneously, packed together into a matrix Q. The keys and values are also packed together into matrices K and V. The matrix of outputs is computed as Equation (1).

The two most commonly used attention functions are additive attention and dot-product attention. Dot-product attention is identical to our algorithm, except for the scaling factor of sqrt(d_k). Additive attention computes the compatibility function using a feed-forward network with a single hidden layer. While the two are similar in theoretical complexity, dot-product attention is much faster and more space-efficient in practice, since it can be implemented using highly optimized matrix multiplication code. While for small values of d_k the two mechanisms perform similarly, additive attention outperforms dot-product attention without scaling for larger values of d_k. We suspect that for large values of d_k the dot products grow large in magnitude, pushing the softmax function into regions where it has extremely small gradients. To counteract this effect, we scale the dot products by 1/sqrt(d_k).

$$\text{Attention}(Q, K, V) = \text{softmax}(Q K^T / \sqrt{d_k}) V \quad (1)$$

3.2.2 Multi-Head Attention

Instead of performing a single attention function with d_model-dimensional keys, values and queries, we found it beneficial to linearly project the queries, keys and values h times with different, learned linear projections to d_k, d_k and d_v dimensions, respectively. On each of these projected versions of queries, keys and values we then perform the attention function in parallel, yielding d_v-dimensional output values. These are concatenated and once again projected, resulting in the final values, as described in Equation (2). Multi-head attention allows the model to jointly attend to information from different representation subspaces at different positions.

With a single attention head, averaging inhibits this. In this work we employ h = 8 parallel attention layers, or heads. For each of these we use d_k = d_v = d_model / h = 64. Due to the reduced dimension of each head, the total computational cost is similar to that of single-head attention with full dimensionality. The projections are parameter matrices W_i^Q, W_i^K, W_i^V and W^O, and the concatenation of the eight heads is followed by a final linear projection.

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, ..., \text{head}_h) W^O \quad (2)$$

3.2.3 Applications of Attention in our Model

The Transformer uses multi-head attention in three different ways. In encoder-decoder attention layers, the queries come from the previous decoder layer, and the memory keys and values come from the output of the encoder. This allows every position in the decoder to attend over all positions in the input sequence. The encoder contains self-attention layers in which all of the keys, values and queries come from the same place, in this case, the output of the previous layer in the encoder. Each position in the encoder can attend to all positions in the previous layer of the encoder. Similarly, self-attention layers in the decoder allow each position in the decoder to attend to all positions in the decoder up to and including that position. We need to prevent leftward information flow in the decoder to preserve the auto-regressive property, which we implement by masking out all values in the softmax input corresponding to illegal connections.

3.3 Position-wise Feed-Forward Networks

In addition to attention sub-layers, each of the layers in our encoder and decoder contains a fully connected feed-forward network, which is applied to each position separately and identically. This consists of two linear transformations with a ReLU activation in between, as in Equation (3). While the linear transformations are the same across different positions, they use different parameters from layer to layer. Another way of describing this is as two convolutions with kernel size 1. The dimensionality of input and output is d_model = 512, and the inner-layer has dimensionality d_ff = 2048.

$$\text{FFN}(x) = \max(0, x W_1 + b_1) W_2 + b_2 \quad (3)$$

3.4 Embeddings and Softmax

Similarly to other sequence transduction models, we use learned embeddings to convert the input tokens and output tokens to vectors of dimension d_model. We also use the usual learned linear transformation and softmax function to convert the decoder output to predicted next-token probabilities. In our model, we share the same weight matrix between the two embedding layers and the pre-softmax linear transformation. In the embedding layers, we multiply those weights by sqrt(d_model). The resulting model is compact: the base configuration has 65 million trainable parameters, while the larger configuration described in Section 6.2 has 213 million.

3.5 Positional Encoding

Since our model contains no recurrence and no convolution, in order for the model to make use of the order of the sequence, we must inject some information about the relative or absolute position of the tokens in the sequence. To this end, we add positional encodings to the input embeddings at the bottoms of the encoder and decoder stacks. The positional encodings have the same dimension d_model as the embeddings, so that the two can be summed. There are many choices of positional encodings, learned and fixed.

In this work, we use sine and cosine functions of different frequencies, as given in Equation (4), where pos is the position and i is the dimension. That is, each dimension of the positional encoding corresponds to a sinusoid. The wavelengths form a geometric progression from 2 pi to 10000 times 2 pi. We chose this function because we hypothesized it would allow the model to easily learn to attend by relative positions, since for any fixed offset k, PE(pos + k) can be represented as a linear function of PE(pos). We also experimented with using learned positional embeddings instead, and found that the two versions produced nearly identical results. We chose the sinusoidal version because it may allow the model to extrapolate to sequence lengths longer than the ones encountered during training.

$$\text{PE}(pos, 2i) = \sin(pos / 10000^{(2i / d_model)}) \quad (4)$$

4 Why Self-Attention

In this section we compare various aspects of self-attention layers to the recurrent and convolutional layers commonly used in mapping a variable-length sequence of symbol representations to another sequence of equal length, such as a hidden layer in a typical sequence transduction encoder or decoder. Motivating our use of self-attention we consider three desiderata. First, the total computational complexity per layer. Second, the amount of computation that can be parallelized, as measured by the minimum number of sequential operations required. Third, the path length between long-range dependencies in the network, since one of the key factors affecting the ability to learn long-range dependencies is the length of the paths forward and backward signals have to traverse.

As side benefit, self-attention could yield more interpretable models. We inspect attention distributions from our models and present and discuss examples in the appendix. Not only do individual attention heads clearly learn to perform different tasks, many appear to exhibit behavior related to the syntactic and semantic structure of the sentences. A single self-attention layer connects all positions with a constant number of sequentially executed operations, whereas a recurrent layer requires O(n) sequential operations. In terms of computational complexity, self-attention layers are faster than recurrent layers when the sequence length n is smaller than the representation dimensionality d, which is the case for the state-of-the-art models used in machine translation.

5 Training

This section describes the training regime for our models. We trained on the standard WMT 2014 English-German dataset consisting of about 4.5 million sentence pairs, and on the WMT 2014 English-French dataset consisting of 36 million sentences. Sentences were encoded using byte-pair encoding, which has a shared source-target vocabulary of about 37000 tokens for English-German and 32000 tokens for English-French.

5.1 Training Data and Batching

We trained on the standard WMT 2014 English-German dataset consisting of about 4.5 million sentence pairs. Sentences were encoded using byte-pair encoding, which has a shared source-target vocabulary of about 37000 tokens. For English-French, we used the significantly larger WMT 2014 English-French dataset consisting of 36 million sentences and split tokens into a 32000 word-piece vocabulary. Sentence pairs were batched together by approximate sequence length. Each training batch contained a set of sentence pairs containing approximately 25000 source tokens and 25000 target tokens.

5.2 Hardware and Schedule

We trained our models on one machine with 8 NVIDIA P100 GPUs. For our base models using the hyperparameters described throughout the paper, each training step took about 0.4 seconds. We trained the base models for a total of 100,000 steps or 12 hours. For our big models, described in Section 6.2, step times were 1.0 seconds, and the big models were trained for 300,000 steps, which corresponds to 3.5 days. Training the big model therefore costs roughly seven times the wall-clock time of the base model while using the same eight GPUs.

We used the Adam optimizer with beta_1 = 0.9, beta_2 = 0.98 and epsilon = 1e-9, and we varied the learning rate over the course of training according to the schedule described in Section 5.3. The checkpoints were averaged to produce the final model, and decoding used beam search with a beam size of 4 and length penalty 0.6.

5.3 Optimizer

We used the Adam optimizer with beta_1 = 0.9, beta_2 = 0.98 and epsilon = 1e-9. We varied the learning rate over the course of training according to Equation (5), increasing it linearly for the first 4000 training steps and decreasing it thereafter proportionally to the inverse square root of the step number. We used a warmup of 4000 steps in all of our experiments. This schedule was chosen because it kept the effective step size stable early in training, when gradient magnitudes are poorly estimated, and then decayed smoothly.

We also experimented with a fixed learning rate and with cosine decay, but both degraded BLEU by more than 0.4 points on the development set. The inverse-square-root schedule was stable across model sizes, and the same beta_1, beta_2 and epsilon values were used for the base and the big models.

$$\text{lrate} = d_{\text{model}}^{-0.5} * \min(\text{step}^{-0.5}, \text{step} * \text{warmup}^{-1.5}) \quad (5)$$

5.4 Regularization

We employ three types of regularization during training: residual dropout, attention dropout and label smoothing. We apply dropout to the output of each sub-layer, before it is added to the sub-layer input and normalized. Additionally, we apply dropout to the

sums of the embeddings and the positional encodings in both the encoder and decoder stacks. For the base model we use a rate of P_drop = 0.1, and we did not apply dropout in the big model because it underfit at that scale.

During training, we employed label smoothing of value epsilon_ls = 0.1. This hurt perplexity, as the model learns to be more unsure, but improves accuracy and BLEU score. Dropout and label smoothing were the two regularization choices that mattered most in our ablations: removing either of them reduced the final BLEU score by roughly one point on newstest2014.

6 Results

On the WMT 2014 English-to-German translation task, the big transformer model outperforms the best previously reported models, including ensembles, by over 2.0 BLEU, establishing a new state-of-the-art BLEU score of 28.4. On the WMT 2014 English-to-French translation task, our big model achieves a BLEU score of 41.8, outperforming all of the previously published single models at a fraction of the training cost of any of the competitive models.

6.1 Machine Translation

On the WMT 2014 English-to-German translation task, the big transformer model outperforms the best previously reported models, including ensembles, by over 2.0 BLEU, establishing a new state-of-the-art BLEU score of 28.4 on newstest2014. The base model also surpasses all previously published models and ensembles, at a fraction of the training cost of any of the competitive models. Table 1 reports the BLEU scores of the base and big configurations together with the strongest published baselines.

On the WMT 2014 English-to-French translation task, our big model achieves a BLEU score of 41.8, outperforming all of the previously published single models, at less than one quarter of the training cost of the previous state-of-the-art model. The English-to-French results confirm that the gains are not specific to a single language pair. All BLEU scores were computed on newstest2014 with a single model and no ensembling, and checkpoints were averaged over the last ten epochs.

Table 1: BLEU scores on newstest2014. The Transformer (big) uses beam search with beam size 4.

Model	EN-DE	EN-FR
Transformer (base)	27.3	38.1
Transformer (big)	28.4	41.8
ConvS2S ensemble	26.30	41.29
GNMT + RL ensemble	26.36	41.16

6.2 Model Variations

To evaluate the importance of different components of the Transformer, we varied our base model in different ways, measuring the change in performance on English-to-German translation on the development set, newstest2013. Table 2 summarizes these variations. As expected, removing a component hurts quality, and the number of heads and the model width interact strongly. While single-head attention is 0.9 BLEU worse than the best setting, quality also drops off with too many heads.

Reducing the attention key size d_k hurts model quality, which suggests that determining compatibility is not easy and that a more sophisticated compatibility function than dot product may be beneficial. We further observe that bigger models are better, and dropout is very helpful in avoiding over-fitting at the base scale. Replacing the sinusoidal positional encoding with learned embeddings produces nearly identical results, which indicates that the model is robust to this design choice.

Table 2: Variations of the Transformer on newstest2013 English-German development set.

Variation	BLEU
base (N=6, d=512, h=8)	27.3
N = 6, d_model = 1024	27.4
h = 16 heads	27.4
dropout 0.0	25.9
label smoothing 0.0	25.6
learned positional embedding	27.2

7 Conclusion

In this work, we presented the Transformer, the first sequence transduction model based entirely on attention, replacing the recurrent layers most commonly used in encoder-decoder architectures with multi-headed self-attention. For translation tasks, the Transformer can be trained significantly faster than architectures based on recurrent or convolutional layers. On both WMT 2014 English-to-German and WMT 2014 English-to-French translation tasks, we achieve a new state of the art. In the former task our best model outperforms even all previously reported ensembles.

We are excited about the future of attention-based models and plan to apply them to other tasks. We plan to extend the Transformer to problems involving input and output modalities other than text and to investigate local, restricted attention mechanisms to efficiently handle large inputs and outputs such as images, audio and video. Making generation less sequential is another research goal. The code we

used to train and evaluate our models is available in the Tensor2Tensor package, and we welcome reproductions on other hardware.

References

[1] Bahdanau, Cho, and Bengio. Neural Machine Translation by Jointly Learning to Align and Translate. ICLR, 2015.

[2] Sutskever, Vinyals, and Le. Sequence to Sequence Learning with Neural Networks. NeurIPS, 2014.

[3] Cho et al. Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation. EMNLP, 2014.

[4] Hochreiter and Schmidhuber. Long Short-Term Memory. Neural Computation, 1997.

[5] Kingma and Ba. Adam: A Method for Stochastic Optimization. ICLR, 2015.

[6] Gehring et al. Convolutional Sequence to Sequence Learning. ICML, 2017.

[7] Wu et al. Google's Neural Machine Translation System: Bridging the Gap between Human and Machine Translation. arXiv preprint, 2016.

[8] Ba, Kiros, and Hinton. Layer Normalization. arXiv preprint, 2016.

[9] Srivastava et al. Dropout: A Simple Way to Prevent Neural Networks from Overfitting. JMLR, 2014.

[10] Szegedy et al. Rethinking the Inception Architecture for Computer Vision. CVPR, 2016.

[11] He et al. Deep Residual Learning for Image Recognition. CVPR, 2016.

[12] Krizhevsky, Sutskever, and Hinton. ImageNet Classification with Deep Convolutional Neural Networks. NeurIPS, 2012.

A Appendix: Training Details

All models were trained on a single machine with 8 NVIDIA P100 GPUs, each with 16GB of memory. The base model used a batch of approximately 25000 source tokens and 25000 target tokens and was trained for 100,000 steps, which took 12 hours. The big model used a batch size of 25000 tokens as well, but step times were roughly 1.0 seconds and training ran for 300,000 steps, or 3.5 days. We used the Adam optimizer with beta_1 = 0.9, beta_2 = 0.98, epsilon = 1e-9 and a warmup of 4000 steps, and we applied residual dropout with rate 0.1 and label smoothing with value 0.1 for the base model.

Decoding used beam search with a beam size of 4 and length penalty 0.6 on newstest2014. We averaged the last ten checkpoints to produce the final model, and reported single-model BLEU without ensembling. Learning rate schedules and dropout rates were selected on newstest2013, and the test sets were only evaluated once for the final configuration. These settings are shared by all translation results reported in the paper.